

NoppaKauppa

Paperinen työpaketti

Lautapeliien verkkokauppa: Vue 3, Pinia, FastAPI, SQLite ja julkaisu tuotantoon

18 työviikkoa · päivätön aikataulu, viikot 1–18 · luovutus 18. työviikon perjantai

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu on päivätön: viikot ovat työviikkoja 1–18 opiskelijan omasta aloituksesta, eivät kalenteriviikkoja.

Julkiseen repositoryyn ei laiteta henkilötietoja, salaisuuksia eikä oikeiden kauppojen tekstejä tai kuvia. Tuotedata keksitään itse.

Aikataulu

Vko	Ajoitus	Viikon aihe	Vaihe
1	Työviikko 1 / 18	AloitUS: toimeksianto ja työympäristö	A
2	Työviikko 2 / 18	Suunnitelma ja tietomalli	A
3	Työviikko 3 / 18	Tuotelista kannasta	A
4	Työviikko 4 / 18	Kategoriat ja reititys	A
5	Työviikko 5 / 18	Ensijulkaisu	A
6	Työviikko 6 / 18	Sanahaku	B
7	Työviikko 7 / 18	Ostoskori	B
8	Työviikko 8 / 18	Tunnukset ja istunto	B
9	Työviikko 9 / 18	Tilaus ja tilaushistoria	B
10	Työviikko 10 / 18	Asiakaskatselmointi	B
11	Työviikko 11 / 18	Palautemuutos ja omat tiedot	C
12	Työviikko 12 / 18	Tuotehallinta ja roolisuojaus	C
13	Työviikko 13 / 18	Tilaukset ja saavutettavuus	C
14	Työviikko 14 / 18	Tietoturvaviikko	C
15	Työviikko 15 / 18	Testaus ja regressio	C
16	Työviikko 16 / 18	Julkaisuehdokas ja julkaisutestaus	D
17	Työviikko 17 / 18	Julkaisu v1.0	D
18	Työviikko 18 / 18	Näyttö	D

Palautus viimeistään 18. työviikon perjantai. Sivusto: tehtävien tarkat ohjeet, toteutusavut ja projektipäiväkirja.

Vaihe A – Kaupan ydin: toimeksianto, tietomalli ja julkaistu tuotelista

Työviikko 1 / 18 – Aloitus: toimeksianto ja työympäristö

Viikon jälkeen toimeksianto on luettu ja kysymyslista viety ohjaajalle: repository on GitHubissa ensimmäisellä commitilla, ja sekä Vue+Vite-frontend että FastAPI-backend käynnistyvät omalla koneellasi.

- ☐ Lue toimeksianto ja kirjaa vähintään kuusi kysymystä ohjaajalle tai asiakkaalle.
- ☐ Sovi julkisen repositoryn asiat ja käynnistä ohjaajan kanssa katselmoijien etsintä viikkoja 10 ja 16 varten.
- ☐ Luo repository ja kansiorakenne: frontend/, backend/ ja project-docs/.
- ☐ Alusta Vite+Vue-projekti ja FastAPI-sovellus ja käynnistä molemmat kehityspalvelimet.

Valmis kun: Molemmat kehityspalvelimet käynnistyvät ohjeen komennoina; repository on GitHubissa README:n ja ensimmäisen commitin kanssa; kysymyslista on ohjaajalla ja vastaukset tai avoimet asiat, mukaan lukien katselmoijien tilanne, on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: repositoryn linkki, ensimmäisen commitin tunnus ja kysymyslista project-docs-kansiossa.

Työviikko 2 / 18 – Suunnitelma ja tietomalli

Viikon jälkeen tekninen suunnitelma on olemassa: käyttäjätarinat priorisoituina pakolliseen ytimeen (P0) ja sen jatkokon (P1, P2), SQLite-tietomalli, tietovarastovertailu, rautalangat ja P0-tehtävät GitHub-issueina, ja teemapäätös on lyöty lukkoon.

- ☐ Kirjoita käyttäjätarinat toimeksiannosta ja priorisoi ne P0-, P1- ja P2-tasolle.
- ☐ Piirrä tietomalli: tuote, kategoria, käyttäjä rooleineen, tilaus ja tilausrivi.
- ☐ Vertaa tietovarastovaihtoehdot (SQLite, JSON-tiedosto, PostgreSQL) ja perustele valinta.
- ☐ Luonnostele rautalangat, hyväksyt ne ja pilko P0 vähintään kahdeksaksi issueksi.

Valmis kun: suunnitelma.md:ssä on tarinat prioriteetteineen, tietomallikaavio, tietovarastoperustelu ja teemapäätös; issueita on vähintään kahdeksan ja P0 on merkitty; rautalankojen hyväksyntä on kirjattu: kuka hyväksyi ja mitä hän sanoi. Hyväksyjäksi käy ohjaaja asiakkaan äänenä, jos katselmoijaa ei ole vielä nimetty.

Työnäyte Git-repositoryyn ennen rastia: suunnitelma.md, tietomallikuva, issue-listan linkki ja hyväksyntämerkintä.

Työviikko 3 / 18 – Tuotelista kannasta

Viikon jälkeen tuotelista tulee SQLite-kannasta FastAPI-rajapinnan kautta ja renderöityy itse rakennetuissa Vue-komponenteissa, lataus- ja virhetilat mukaan lukien.

- ☐ Luo SQLite-kanta ja seed-skripti noin 20 oman teeman tuotteella.
- ☐ Toteuta GET /api/tuotteet ja kokeile se FastAPI:n /docs-sivulla.
- ☐ Rakenna TuoteLista- ja TuoteKortti-komponentit itse ilman valmista käyttöliittymän komponenttikirjastoa.
- ☐ Käsittele lataus-, tyhjä- ja virhetilat sekä kirjaa ja aja testitapaukset T01 ja T02.

Valmis kun: Kehityspalvelin näyttää kannasta tulevat tuotteet; API palauttaa JSONia, mikä on testattu /docs-sivulta; kun backend sammutetaan, käyttäjä näkee virheilmoituksen eikä tyhjää ruutua.

Työnäyte Git-repositoryyn ennen rastia: commit-linkki, kuvakaappaus tuotelistasta ja testitapausten T01–T02 tulokset päiväkirjassa.

Työviikko 4 / 18 – Kategoriat ja reititys

Viikon jälkeen vue-router on käytössä: kategorianäkymä, tuotesivu omalla osoitteellaan (URL) ja 404-näkymä. Navigointi toimii puhelimella.

- ☐ Asenna ja konfiguroi vue-router ja kirjaa, miksi ulkoinen reitityskomponentti tarvitaan.
- ☐ Toteuta kategorialistaus ja kategoriakohtainen tuotenäkymä.
- ☐ Toteuta tuotesivu reittiparametrilla ja 404-reitti.
- ☐ Päätä responsiivisuuden taitekohdat, rakenna saavutettavuuden perusta sisään ja testaa oikealla puhelimella.

Valmis kun: Jokaisella tuotteella ja kategorialla on oma URL, joka toimii myös selaimen päivityksellä; tuntematon osoite näyttää 404-näkymän; navigointi toimii omalla puhelimella ja perusnäkymissä pääsee liikkumaan näppäimistöllä.

Työnäyte Git-repositoryyn ennen rastia: commit, reittikartta suunnitelmassa ja kuvakaappaus omalta puhelimelta.

Työviikko 5 / 18 – Ensijulkaisu

Viikon jälkeen NoppaKauppa on julkisessa osoitteessa (frontend, backend ja kanta), ja julkaisualustan valinta on vertailtu ja perusteltu.

- ☐ Vertaa vähintään kolme julkaisuvaihtoehtoa ja valitse perustellen.
- ☐ Konfiguroi tuotanto-build ja ympäristömuuttujat; .env ei mene repositoryyn.
- ☐ Julkaise kauppa ja savutestaa julkinen osoite toisella laitteella (T03).
- ☐ Aloita käyttöönotto-ohje READMEen.

Valmis kun: Julkinen URL näyttää tuotelistan kannasta; osoite toimii puhelimella ja toisella koneella; salaisuudet ja .env-tiedosto eivät ole repositoryssä.

Työnäyte Git-repositoryyn ennen rastia: julkinen URL, julkaisulokin kuvakaappaus, vertailutaulukko suunnitelmassa ja T03:n tulos.

Vaihe B – Ostopolku: haku, kori, tunnukset, tilaus ja asiakaskatselmointi

Työviikko 6 / 18 – Sanahaku

Viikon jälkeen sanahaku löytää tuotteet nimen osalla kirjainkoosta riippumatta; tyhjä tulos ja backend-virhe on käsitelty, ja kysely on parametrisoitu.

- ☐ Toteuta haku-endpoint kyselyparametrilla parametrisoidulla SQL:llä ja kirjaa, miksi parametrisointi tehdään.
- ☐ Rakenna hakukenttä ja tuloslista.
- ☐ Käsittele tyhjä tulos opastavalla tekstillä ja backend-virhe omalla viestillään.
- ☐ Kirjaa ja aja testitapaukset T04 ja T05.

Valmis kun: Haku "nop" löytää "Nopanheitto"-tuotteen; haku merkkijonolla ' DROP TABLE palauttaa nolla osumaa eikä kaada mitään; tyhjä tulos opastaa käyttäjää.

Työnäyte Git-repositoryyn ennen rastia: commit, testitapausten T04–T05 tulokset ja kuvakaappaus hakudemosta.

Työviikko 7 / 18 – Ostoskori

Viikon jälkeen Pinia-ostoskori toimii: lisää, poista, muuta määrää, summa laskee oikein ja kori säilyy sivun päivityksen yli, ja koko ominaisuus syntyi omassa haarassa eli erillisessä kehityslinjassa ja yhdistyi pääversioon pull requestilla.

- ☐ Vertaa korin tallennusvaihtoehdot ja tee päätös ohjaajan kanssa kirjaten.
- ☐ Toteuta kori-store haarassa feature/ostoskori ja yhdistä se pull requestilla.
- ☐ Rakenna korinäkömää määränmuutoksineen ja summineen.
- ☐ Kirjaa ja aja testitapaukset T06 ja T07 sekä kirjaa virheenkorjausketju K1.

Valmis kun: Tuotteita voi lisätä ja poistaa, määrää muuttaa ja summa täsmää; kori säilyy sivun päivityksessä; pull request on yhdistetty ja mahdollinen konflikti on ratkaistu.

Työnäyte Git-repositoryyn ennen rastia: pull requestin linkki, päätösmuistio, testitapausten T06–T07 tulokset ja K1-ketju päiväkirjassa.

Työviikko 8 / 18 – Tunnukset ja istunto

Viikon jälkeen rekisteröityminen ja kirjautuminen toimivat: salasanat tallennetaan vain hashattuna, istunto hoidetaan perustellulla ratkaisulla, ja käyttäjällä on roolikenttä tulevia henkilökuntanäkymiä varten.

- ☐ Vertaa istuntoratkaisut (JWT ja eväste-sessio) ja kirjaa päätös perusteluineen ohjaajan kommentilla.
- ☐ Toteuta rekisteröityminen validointineen ja salasanan hashaus.
- ☐ Toteuta kirjautuminen, uloskirjautuminen ja auth-store Piniaan.
- ☐ Kirjaa ja aja testitapaukset T08 ja T09.

Valmis kun: Kannassa ei näy yhtään selkokielistä salasanaa; väärä salasana ja tuntematon tunnus antavat saman yleisen virheilmoituksen; kirjautumistila säilyy sivun päivityksessä ja uloskirjautuminen tyhjentää sen.

Työnäyte Git-repositoryyn ennen rastia: päätösmuistio, kuvakaappaus kantariveistä (hash näkyy, salasana ei) ja testitapausten T08–T09 tulokset.

Työviikko 9 / 18 – Tilaus ja tilaushistoria

Viikon jälkeen kirjautunut asiakas tilaa korin sisällön ilman maksunvälitystä: tilaus tallentuu riveineen tilaushetken hinnoilla, kori tyhjenee ja tilaus näkyy omassa tilaushistoriassa.

- ☐ Toteuta tilauksen luonti transaktiona: tilaus ja rivit tallentuvat yhtenä kokonaisuutena.
- ☐ Rakenna tilauslomake ja valitse validointiin perusteltu ulkoinen paketti tai perustele oma validointi.
- ☐ Toteuta tilaushistorianäkymä, joka näyttää vain omat tilaukset.
- ☐ Kirjaa ja aja testitapaukset T10 ja T11.

Valmis kun: Tilaus näkyy kannassa riveineen ja tilaushetken hinnoilla; historia näyttää vain omat tilaukset; tyhjää koria ei voi tilata eikä kirjautumaton pääse tilaamaan.

Työnäyte Git-repositoryyn ennen rastia: commit, lisäpaketin perustelu suunnitelmassa, testitapausten T10–T11 tulokset ja kuvakaappaus historiasta.

Työviikko 10 / 18 – Asiakaskatselmointi

Viikon jälkeen ulkopuolinen henkilö asiakkaan roolissa on kokeillut julkaistua kauppaa tehtäväradalla (löydä peli, koriin, tilaa), palaute on kirjattu hänen omin sanoin erillään omasta tulkinnasta, ja muutokset on sovittu issueiksi.

- ☐ Kutsu nimetty ulkopuolinen asiakkaan rooliin ja laadi kolmen tehtävän tehtävärata.
- ☐ Vedä katselmointi julkisessa osoitteessa ja kirjaa sitaatit ja oma tulkinta erikseen.
- ☐ Muuta palaute issueiksi ja priorisoi ne ohjaajan kanssa.
- ☐ Kirjoita asiakkaalle yhteenveto ilman teknistä jargonia.

Valmis kun: Katselmointimuistiossa on nimetty rooli, testaajan omat sanat ja niistä erillään oma tulkinta; vähintään kolme palautekohtaa on issueina prioriteetteineen; asiakasyhteenveto on kirjoitettu.

Työnäyte Git-repositoryyn ennen rastia: katselmointimuistio project-docs-kansiossa ja palauteissueiden linkit.

Vaihe C – Valmiiksi: palautemuutos, henkilökunnan työkalut, tietoturva ja laatu

Työviikko 11 / 18 – Palautemuutos ja omat tiedot

Viikon jälkeen tärkein katselmointipalaute on toteutettu näkyväksi muutokseksi tuotantoon asti, asiakastietojen muokkaus on valmis, ja suunnitelman työmääräarviot on päivitetty toteutuneen perusteella.

- ☐ Toteuta katselmoinnin tärkein muutos ketjuna: issue, haara, commit ja tuotanto.
- ☐ Toteuta omien asiakastietojen muokkaus; käyttäjätunnusta ei voi vaihtaa.
- ☐ Päivitä suunnitelma ja työmääräarviot toteutuneen perusteella.
- ☐ Kirjaa virheenkorjausketju K2 aidosta havainnosta.

Valmis kun: Muutos näkyy julkaistussa versiossa ja issue sulkeutuu committiin viitaten; omia tietoja voi muokata mutta käyttäjätunnusta ei; suunnitelmassa näkyy päivitetty arvio perusteluineen.

Työnäyte Git-repositoryyn ennen rastia: ketju issuesta committiin ja tuotantolinkkiin, K2-ketju ja suunnitelman muutoshistoria.

Työviikko 12 / 18 – Tuotehallinta ja roolisuojaus

Viikon jälkeen henkilökuntarooli hallitsee tuotteita selaimessa (lisäys, muokkaus ja poisto), ja hallintareitit on suojattu kahdella tasolla: reitin vartija (route guard) frontendissä ja roolitarkistus API:ssa.

- ☐ Toteuta roolipohjainen pääsynhallinta: reitin vartija (route guard) frontendissä ja roolitarkistus rajapinnassa.
- ☐ Rakenna tuotehallintanäkymä lisäys-, muokkaus- ja poistolomakkeineen.
- ☐ Validoi syötteet sekä frontendissä että backendissä.
- ☐ Kirjaa ja aja testitapaukset T12 ja T13, joissa asiakasrooli yrittää hallintaan myös suoralla rajapintakutsulla.

Valmis kun: Asiakasrooliilla suora URL hallintanäkymään ohjaa pois ja suora rajapintakutsu palauttaa virhekoodin 401 (ei kirjautunut) tai 403 (ei oikeuksia); henkilökunnan lisäämä tuote näkyy heti kaupan puolella.

Työnäyte Git-repositoryyn ennen rastia: testitapausten T12–T13 tulokset curl-kutsun vastauksineen, commit ja kuvakaappaus hallintanäkymästä.

Työviikko 13 / 18 – Tilaukset ja saavutettavuus

Viikon jälkeen henkilökunta selaa kaikkia tilauksia uusimmat ensin ja merkitsee tilauksen käsitellyksi, ja tilan muutos näkyy asiakkaan tilaushistoriassa. Ominaisuuslista päättyy tähän; viikon toinen puolisko on koko sovelluksen saavutettavuustarkistus.

- ☐ Toteuta henkilökunnan tilauslistaus suodatuksella: uudet ja käsitellyt.
- ☐ Toteuta tilauksen tila ja sen muutos, näytä tila asiakkaan historiassa ja aja testitapaus T14.
- ☐ Tee saavutettavuustarkistus koko sovellukselle ja korjaa löydökset.
- ☐ Aja Lighthouse ennen ja jälkeen korjausten ja tallenna raportit.

Valmis kun: Henkilökunta näkee kaikkien tilaukset ja asiakas vain omansa; käsitellyksi-merkintä näkyy asiakkaalle; ostopolun voi kulkea läpi pelkällä näppäimistöllä; Lighthouse-raportit ennen ja jälkeen on tallennettu ja vähintään yksi saavutettavuuspuute on korjattu.

Työnäyte Git-repositoryyn ennen rastia: commit, T14:n tulos, kuvakaappaukset molemmilla rooleilla ja Lighthouse-raportit ennen ja jälkeen.

Työviikko 14 / 18 – Tietoturvaviikko

Viikon jälkeen kirjallinen tietoturva-arvio on olemassa (uhkat, ratkaisut ja jäännösriskit), ja vähintään kaksi kovennusta on toteutettu omien hyökkäystestien löydösten perusteella.

- ☐ Käy sovellus läpi tietoturvan tarkistuslistalla kohta kohdalta.
- ☐ Testaa hyökkääjän silmin: SQL-injektio, XSS, IDOR ja hallinnan rajapinta ilman istuntoa.
- ☐ Korjaa löydökset ja kirjaa virheenkorjausketju K3.
- ☐ Kirjoita tietoturva-arvio.md ja käy se läpi ohjaajan kanssa.

Valmis kun: tietoturva-arvio.md:ssä jokaisella tunnistetulla riskillä on ratkaisu tai perusteltu jäännösriski; vähintään yksi aito löydös on korjattu täydellisenä K3-ketjuna; ohjaajan läpikäynti on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: tietoturva-arvio.md, hyökkäystestien kirjaukset, K3-ketju ja korjauscommitit.

Työviikko 15 / 18 – Testaus ja regressio

Viikon jälkeen koko testimatriisi eli neljätoista tapausta on ajettu julkaistua versiota vasten tuloksineen (mukaan lukien viikkojen 13 ja 14 korjausten regressiot), ja koodia on refaktoroitu vähintään yhdessä, enintään kahdessa kohdassa käyttäytymistä muuttamatta.

- ☐ Aja koko testimatriisi T01–T14 julkaistua versiota vasten ja kirjaa jokaisen rivin toteutunut tulos.
- ☐ Aja viikkojen 13 ja 14 korjausten regressiotestit.
- ☐ Refaktoroi vähintään yksi, enintään kaksi kohdetta ja aja matriisin osumat uudelleen.

Valmis kun: Matriisin jokaisella rivillä on odotettu ja toteutunut tulos julkaistua versiota vasten; refaktorointicommitteissa testit on ajettu uudelleen ja tulos kirjattu; poikkeamat ovat issueina.

Työnäyte Git-repositoryyn ennen rastia: testimatriisin tulostaulukko project-docs-kansiossa, refaktorointicommitit ja poikkeamaissuet.

Vaihe D – Julkaisu ja näyttö

Työviikko 16 / 18 – Julkaisuehdokas ja julkaisutestaus

Viikon jälkeen sisältö on jäädytetty ja julkaisuehdokas (release candidate, RC) v1.0-rc1 on julkaistu, ja ulkopuolinen henkilö on ottanut kaupan käyttöön ja tehnyt testitilauksen puhtaassa ympäristössä pelkän kirjoitetun ohjeen avulla, ilman suullista apua.

- ☐ Jäädytä sisältö, julkaise julkaisuehdokas (RC) ja merkitse git-tag v1.0-rc1.
- ☐ Siivoa repository ja viimeistele käyttöönotto- ja käyttöohje.
- ☐ Järjestä julkaisutestaus: nimetty ulkopuolinen ja puhdas ympäristö ilman suullista apua.
- ☐ Kirjaa havainnot testaajan sanoin ja luokittele estävät ja ei-estävät löydökset.

Valmis kun: Testaaja pääsi ohjeen avulla rekisteröitymisestä testitilaukseen asti ilman suullista apua, tai jokainen epärointikohta on kirjattu ohjeen korjauslistaksi; estävät virheet on listattu erikseen.

Työnäyte Git-repositoryyn ennen rastia: v1.0-rc1-tag, julkaisutestin pöytäkirja ja ohjeen korjauslista.

Työviikko 17 / 18 – Julkaisu v1.0

Viikon jälkeen estävät virheet on korjattu (vain ne, jäädytys pitää), v1.0 on julkaistu tuotantoon ja luovutettu asiakkaalle selkokielellä luovutusviestillä, ja dokumentaatio on valmis.

- ☐ Korjaa julkaisutestauksen estävät löydökset; ei-estävät kirjataan tunnetuiksi puutteiksi.
- ☐ Julkaise versio 1.0 ja merkitse git-tag.
- ☐ Viimeistele dokumentaatio: käyttöönotto, käynnistys, riippuvuudet, ympäristömuuttujat ja tunnetut puutteet.
- ☐ Kirjoita asiakkaan luovutusviesti selkokielellä.

Valmis kun: Tagi v1.0 on olemassa; julkinen osoite toimii; README:n avulla ulkopuolinen saa kehitysympäristön käyntiin; luovutusviesti ja tunnetut puutteet on kirjattu.

Työnäyte Git-repositoryyn ennen rastia: v1.0-tag ja julkaisumuistio, tuotanto-URL sekä luovutusviesti project-docs-kansiossa.

Työviikko 18 / 18 – Näyttö

Viikon jälkeen mitään uutta ei ole lisätty: kaikkien 32 vaatimuksen työnäytteet on täsmälinkitetty näyttömatriisiin, 8–10 minuutin demo on harjoiteltu ja itsearviointi on kirjoitettu.

- ☐ Linkitä kaikkien 32 vaatimuksen työnäytteet näyttömatriisiin.
- ☐ Harjoittele 8–10 minuutin demo ja aja se kellon kanssa.
- ☐ Kirjoita itsearviointi konkreettisin esimerkein.
- ☐ Luovuta aineisto sovitulla tavalla.

Valmis kun: Matriisin jokaisella rivillä on toimiva linkki; demo on ajettu kellon kanssa vähintään kerran toiselle ihmiselle; itsearviointi nimeää konkreettisia tilanteita.

Työnäyte Git-repositoryyn ennen rastia: täytetty näyttömatriisi, demorunko ja itsearviointi päiväkirjan viimeisenä merkintänä.

Viimeiset viisi päivää

Päivä 1 · Sisältöjäädytys: viimeinen hyväksytty versio

Päivä 2 · Aineisto: päiväkirja, testimatriisi, muistiot ja linkit

Päivä 3 · Harjoittelu: 8–10 minuutin demo ja itsearviointi

Päivä 4 · Puskuri ja tarkistus toisen henkilön kanssa

Päivä 5 · Luovutus: näyttömatriisi linkitetty, demo harjoiteltu, aineisto luovutettu

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

frontend vko 1	<i>Selaimessa toimiva osa.</i> Se osa kaupasta, joka toimii asiakkaan selaimessa: näkymät, komponentit ja ulkoasu. Tässä projektissa frontend on kansiossa frontend/ ja se tehdään Vue 3:lla.
backend vko 1	<i>Palvelimella toimiva osa.</i> Se osa kaupasta, joka toimii palvelimella: tietokanta, rajapinta ja käyttöoikeudet. Tässä projektissa backend on kansiossa backend/ ja se tehdään FastAPI:lla.
repository vko 1	<i>Projektin Git-varasto.</i> Kansio, jota Git-versionhallinta seuraa ja joka viedään GitHubiin. Repositoryssä on koko projektin historia, ei vain uusin versio.
commit vko 1	<i>Yksi tallennettu muutos.</i> Yksi muutos tallennettuna Gitin historiaan: mitä muuttui, milloin ja kenen tekemänä. Tässä projektissa tehdään pieniä committeja usein, ei yhtä isoa viikon lopussa.
GitHub-issue vko 2	<i>Yksi rajattu tehtävä.</i> GitHubiin kirjattu tehtävä, jossa on otsikko, kuvaus ja valmis kun -ehto. Yksi issue on noin puolen tai yhden päivän työ, ja commit voi viitata siihen numerolla, esimerkiksi #12.
P0 vko 2	<i>Pakollinen ydin.</i> Ominaisuudet, joiden on valmistuttava, tai kauppa ei ole valmis. P0 tehdään ensin ja kokonaan.
P1 vko 2	<i>Tärkeä jatkosisältö.</i> Ominaisuudet, jotka tehdään vasta kun P0 toimii. P1 on tärkeä, mutta se ei estä julkaisua.
P2 vko 2	<i>Valinnainen lisä.</i> Ominaisuudet, jotka voidaan jättää kokonaan pois. P2 tehdään vain, jos aikaa jää yli.
JSON vko 2	Tekstimuoto, jossa tieto liikkuu selaimen ja palvelimen välillä ja jossa sen voi myös tallentaa tiedostoon. Rajapinta palauttaa tuotteet JSONina.
API vko 3	<i>Rajapinta.</i> Sovittu tapa, jolla frontend pyytää tietoa backendilta ja saa vastauksen. Tämän projektin rajapinta tehdään FastAPI:lla, ja sitä voi kokeilla selaimessa /docs-sivulta.
endpoint vko 3	<i>Rajapinnan yksittäinen osoite.</i> Yksi rajapinnan osoite, joka tekee yhden asian: esimerkiksi GET /api/tuotteet palauttaa tuotelistan. Rajapinta koostuu endpointeista.
T01 vko 3	<i>Testitapauksen tunnus.</i> Testitapaukset numeroidaan: T01 on ensimmäinen ja T14 viimeinen. Jokaisesta kirjataan lähtötila, toiminta, odotettu tulos ennen ajoa ja toteutunut tulos ajon jälkeen.
seed-data vko 3	<i>Kannan aloitusdata.</i> Itse keksitty aloitustavara, joka ajetaan tyhjiin tietokantaan, jotta sovelluksessa on jotain näytettävää. Tässä projektissa noin 20 oman teeman tuotetta.
CORS vko 3	Selaimen sääntö, joka estää sivua hakemasta tietoa toisesta osoitteesta ilman lupaa. Backendiin kirjataan, mistä osoitteista kutsut hyväksytään – tämä on tavallisin syy siihen, että julkaistu kauppa ei saa dataa.
SPA vko 4	<i>Single-page application, yhden sivun sovellus.</i> Sovellus, jonka selain lataa kerran ja joka vaihtaa näkymää itse. Osoitteiden pitää silti toimia myös suoraan avattuna ja sivun päivityksellä.
build vko 5	<i>Tuotantoversion kääntäminen.</i> Komento, joka kääntää frontendin lähdekoodin valmiiksi tiedostoiksi julkaisua varten (Vitessä npm run build, tulos kansioon dist/). Kehityspalvelin ei tee tätä.
SQL vko 6	<i>Tietokannan kyselykieli.</i> Kieli, jolla tietokannasta haetaan ja siihen kirjoitetaan, esimerkiksi SELECT * FROM tuote. SQLite on tietokanta, joka ymmärtää SQL:ää.
SQL-injektio vko 6	Hyökkäys, jossa käyttäjän syöte ujutetaan osaksi tietokantakyselyä ja pääsee muuttamaan sen merkitystä. Estetään parametrisoidulla kyselyllä, jossa syöte ei koskaan liity osaksi kyselytekstiä.
haara vko 7	<i>Branch.</i> Erillinen kehityslinja, jossa yhtä ominaisuutta tehdään rauhassa ilman että pääversio rikkoutuu. Valmis haara yhdistetään takaisin pääversioon.

pull request vko 7	<i>PR, pyyntö yhdistää haara.</i> Pyyntö yhdistää haara pääversioon. Siinä näkyy muutos rivi riviltä, ja se katselmoidaan ennen yhdistämistä. Yksin tehdessä katselmoi oman muutoksesi itse.
K1 vko 7	<i>Virheenkorjausketjun tunnus.</i> Virheenkorjausketjut numeroidaan K1, K2 ja K3. Yksi ketju on kokonaisuus: havainto, toistamisohje, syy, korjauscommit, uusintatesti ja regressiotesti.
regressiotesti vko 7	Testi, jolla varmistetaan, ettei korjaus rikkonut jotain, mikä toimi aiemmin. Se ajetaan korjatun kohdan vierestä, ei itse korjauksesta.
JWT vko 8	<i>JSON Web Token.</i> Allekirjoitettu tunnistetieto, jonka palvelin antaa kirjautumisen jälkeen ja jonka selain lähettää mukana jokaisessa pyynnössä. Vaihtoehto eväste-sessiolle.
hash vko 8	<i>Tiiviste.</i> Salasanasta laskettu merkkijono, josta alkuperäistä salasanaa ei saa takaisin. Kantaan tallennetaan vain hash, ei koskaan salasanaa sellaisenaan.
XSS vko 14	<i>Cross-site scripting.</i> Hyökkäys, jossa käyttäjän syöte päättyy sivulle koodina ja suoritetaan toisen käyttäjän selaimessa. Estetään sillä, ettei syötettä koskaan tulosteta raakana HTML:ksi.
IDOR vko 14	<i>Suojaamaton viittaus tietoon.</i> Aukko, jossa osoitteen tunnusta vaihtamalla näkee toisen käyttäjän tiedot. Estetään tarkistamalla jokaisessa haussa, kuuluuko rivi kirjautuneelle käyttäjälle.
refaktorointi vko 15	Koodin selkeyttäminen niin, että toiminta pysyy täsmälleen ennallaan: parempi nimeäminen, toiston poisto, liian ison komponentin pilkkominen. Testit ajetaan ennen ja jälkeen.
RC vko 16	<i>Release candidate, julkaisuehdokas.</i> Lähes valmis versio, joka jäädytetään ja testataan ennen lopullista julkaisua. RC1 on ensimmäinen julkaisuehdokas; sen jälkeen korjataan vain estävät virheet.
tagi vko 16	<i>Git-tag.</i> Nimilappu, joka merkitsee yhden commitin historiasta, esimerkiksi v1.0-rc1. Tagista tiedetään myöhemmin, mikä versio oli testattavana.
REST	Tapa rakentaa rajapinta niin, että osoite kertoo kohteen ja pyynnön tyyppi tekemisen: GET hakee, POST luo, PUT muuttaa ja DELETE poistaa. Tämän projektin rajapinta noudattaa sitä.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **käyttää ohjelmointieditoria tai kehitysympäristöä** – Viikko 1: kehitysympäristö pystyssä (VS Code, Vite- ja FastAPI-kehityspalvelimet, selaimen kehittäjätyökalut); käynnistyskomennot READMEssä ja päiväkirjassa.
- **etsii ja korjaa virheitä ohjelmakoodista** – Viikot 7, 11 ja 14: kolme täydellistä virheenkorjausketjua (K1 ostoskori, K2 palautemuutos, K3 tietoturvalöydös); havainto, toisto, syy, korjauscommit, uusintatesti ja regressiotesti.
- **testaa ohjelman toimintoja** – Viikko 15: 14 tapauksen testimatriisi (odotetut tulokset kirjattu ennen ajtoa) ajettuna julkaistua versiota vasten sekä uusintatestaukset korjausten jälkeen.
- **käyttää rakenteista ohjelmointia toteutuksissa** – Viikko 9: tilauslogiikka jaettuna moduuleihin ja funktioihin (reitit, kantakerros, validointi); frontendissä komponentti- ja store-jako, ei yhtä jättitiedostoa.
- **kirjoittaa ylläpidettävää ohjelmakoodia** – Viikko 15: refaktorointicommitit perusteluineen; nimeäminen, toisteisuuden poisto ja komponentin pilkkominen, testit vihreinä ennen ja jälkeen.
- **tulkitsee suunnitelmia ja toteuttaa käyttöliittymän tai sen osia** – Viikko 4: näkymät toteutettu viikon 2 hyväksytyistä rautalangoista; responsiivisuuden taitekohdat ja mobiilitestaus oikealla puhelimella.
- **tulkitsee suunnitelmia ja toteuttaa ohjelmiston toimintoja** – Viikko 9: tilaus ja tilaushistoria toteutettu käyttäjätarinoiden ja hyväksymiskriteerien mukaan; kytKentä tarina, issue ja commit.
- **sopii tehtävistä tiimin muiden jäsenten kanssa** – Viikko 2: tehtävät sovittu ohjaajan kanssa ja kirjattu issueiksi; tehtävien tila näkyvänä koko projektin ajan.
- **etsii ratkaisuvaihtoehtoja ja ratkoo ongelmia yhdessä tiimin kanssa** – Viikot 7 ja 8: kaksi kirjattua vertailua ja yhteispäätöstä ohjaajan kanssa (korin tallennus ja istuntoratkaisu).
- **arvioi ratkaisujen toimivuuden yhdessä tiimin kanssa** – Viikko 10: asiakaskatselmoinnin muistio (täyttääkö toteutus tarinat ja mitä muutetaan ennen seuraavaa versiota, priorisoituna ohjaajan kanssa).
- **arvioi omaa toimintaa tiimin jäsenenä** – Viikko 18: itsearviointi konkreettisin tilantein: mikä onnistui, missä tarvitsit apua ja mitä tekisit toisin.

Ohjelmistokehittäjänä toimiminen

- **selvittää kehitystiimin kanssa asiakkaan tarpeet** – Viikko 2: toimeksiannosta johdetut käyttäjätarinat, kysymyslista ja vastaukset sekä raja- ja maksaminen pois) kirjattuna.
- **viestii tekniset asiat asiakaslähtöisesti** – Viikot 10 ja 17: katselmoinnin asiakaskielinen yhteenveto ja luovutusviesti ilman teknistä jargonia; maksurajauksen selitys asiakkaalle.
- **osallistuu version katselmointiin** – Viikot 10 ja 16: kaksi katselmointia (asiakaskatselmointi puolivälissä ja julkaisuehdokkaan julkaisutestaus), muistiot rooleineen ja sitaatteineen.
- **asettaa kehitystiimin kanssa toteutettavat toiminnot tärkeysjärjestykseen** – Viikko 2: käyttäjätarinoiden P0-, P1- ja P2-priorisointi perusteluineen; P0 on toimiva ostopolku ilman maksua.
- **jakaa kehitystiimin kanssa toteutettavat toiminnot tehtäviksi** – Viikko 2: tarinat pilkottu vähintään kahdeksaksi issueksi, joissa yksi issue on noin puolen tai yhden päivän työ.
- **suunnittelee ja arvioi kehitystiimin kanssa tehtävien toteuttamista** – Viikko 11: työmääräarviot ja niiden päivitys toteutuneen perusteella. Suunnitelman muutoshistoria näyttää, mikä arvio muuttui ja miksi.
- **kehittää ohjelmiston toimintalogiikkaa** – Viikot 9 ja 13: tilauksen muodostus transaktiona, tilaushetken hinnat, tilausten tilat ja niiden muutokset sekä roolikohtaiset näkymät.
- **valitsee ohjelmistoon sopivan tietovaraston** – Viikko 2: kirjallinen vertailu SQLite, JSON-tiedosto ja PostgreSQL datan rakenteen, käyttötilanteen ja laajuuden perusteella; perusteltu valinta suunnitelmassa.
- **toteuttaa yhteyden tietovarastoon** – Viikot 3 ja 12: luku (tuotelista, haku) ja kirjoitus (tilaukset, tuotehallinnan lisäys, muokaus ja poisto) SQLite-kantaan hallitusti.
- **hyödyntää rajapintoja ja käsittelee tietoa** – Viikko 6: oma REST-rajapinta (haku kyselyparametrilla, virhetilanteiden käsittely fetchissä sekä lataus-, tyhjä- ja virhetilat käyttöliittymässä).
- **arvioi ohjelmiston tietoturva** – Viikot 8, 12 ja 14: salasanojen hashaus ja istuntopäätös, kaksikerroksinen roolisuojaus sekä tietoturva-arvio omine hyökkäystesteineen (injektio, XSS, IDOR).

- **käyttää versionhallintaa** – Viikko 1 alkaen: Git koko projektin ajan (tarkoituksenmukaiset commitit, etärepository ja sovitut haarakäytännöt); commit-historia näyttöaineistona.
- **liittää ohjelman osan olemassa olevaan versioon** – Viikko 7: ostoskori toteutettu haarassa feature/ostoskori ja yhdistetty pull requestilla; mahdollinen konflikti ratkaistu ja kirjattu.
- **julkaisee ohjelman tuotantoympäristöön** – Viikot 5 ja 17: ensijulkaisu vertailtuun alustaan ympäristömuuttujineen sekä julkaisuehdokkaan ja version 1.0 julkaisut tageineen ja julkaisumuistioineen.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **ottaa käyttöön ja konfiguroi ohjelmistokomponenttikirjaston käyttöön soveltuvan kehittämisympäristön** – Viikot 1 ja 5: Vue 3 - ja Vite-projektin luonti ja konfigurointi; kehitys- ja tuotantoasetusten ero (API-osoite, build) dokumentoituna.
- **selvittää ohjelmistokomponenttikirjaston tarjoamat mahdollisuudet ja rajoitteet** – Viikko 2: selvitys suunnitelmassa siitä, mitä Vue ja Pinia ratkaisevat (komponentit, reaktiivisuus, tila) ja missä tarvitaan muuta (backend rooleille ja istunnoille, yksisivusovelluksen rajoitteet).
- **käyttää ohjelmistokomponenttikirjaston tärkeimpiä toimintoja ja työkaluja** – Viikot 3 ja 7: komponentit, propsit, computed-arvot ja Pinia-store (kori, auth) käytössä; itse rakennettu tuotelistanäkymä ilman valmista käyttöliittymäkirjastoa.
- **tuo kehittämisympäristöön ulkoisia komponentteja** – Viikot 4 ja 9: vue-router perusteltuna sekä yksi oma perusteltu lisäpaketti (käyttötarkoitus, konfigurointi ja riippuvuusvaikutus kirjattuna).
- **suunnittelee, toteuttaa ja testaa ohjelmiston ohjelmistokomponenttikirjastoa käyttäen** – Viikko 15: komponenttirakenne ja vastuut suunnitelmassa, tarinoiden mukainen toteutus ja koko testimatriisi kirjastopohjaista sovellusta vasten.
- **julkaisee ohjelmiston asiakkaan ympäristöön** – Viikko 17: tuotanto-build ja version 1.0 julkaisu sovittuun ympäristöön; luovutus asiakkaalle ohjeineen.
- **dokumentoi ohjelmiston sovitulla tavalla** – Viikko 16: README käyttöönottoineen, käynnistyksineen, riippuvuuksineen, ympäristöasetuksineen ja tunnettuine puutteineen. Ulkopuolisen julkaisutestillä todennettu ohje.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.